



Grado en Inteligencia Artificial
Procesamiento del Lenguaje Natural II
Práctica 2

Curso 2025–2026
Universidad Rey Juan Carlos

Índice

1. Contexto	2
2. Objetivos de aprendizaje	2
3. Material de partida	3
4. Tareas	3
4.1. Módulo de retrieval denso	3
4.2. Clasificador k-NN sobre embeddings	4
4.3. Clasificador híbrido Transformer + k-NN (RAG para clasificación)	5
4.4. Explicabilidad basada en retrieval (ejemplos similares)	5
4.5. Compresión / modelo destilado	6
4.6. Resumen automático como explicación	6
4.7. Análisis global y discusión	8
5. Tarea opcional (para MH)	8
6. Entregables	9
6.1. Informe	9
6.2. Código y notebooks	9
6.3. README	10

1. Contexto

En la **Práctica 1** se ha construido un *pipeline* completo de clasificación de textos en un dominio concreto (sentimiento, toxicidad, noticias, etc.), que incluía:

- Selección y armonización de varios *datasets*.
- Baselines clásicos (por ejemplo, TF-IDF + regresión logística / SVM).
- Fine-tuning de un modelo *Transformer* (p.ej. BERT, RoBERTa, mBERT, XLM-R...).
- Evaluación con métricas adecuadas (Accuracy, Macro-F1, etc.).
- Algún mecanismo básico de explicabilidad (atención, SHAP/LIME, etc.).

La **Práctica 2** parte directamente de ese trabajo y lo extiende en tres direcciones:

- **Retrieval**: construir un módulo de recuperación de ejemplos similares sobre vuestros propios datos.
- **RAG “interpretativo”**: combinar clasificación + retrieval para obtener un sistema que predice y, además, se justifica con ejemplos de entrenamiento.
- **Compresión/destilación + resumen automático**: explorar el impacto de modelos más ligeros y usar resumen abstractivo como herramienta de explicación.

El objetivo es acercarse a algo que se parece más a un *asistente inteligente*: no sólo clasifica, sino que puede enseñar ejemplos relevantes y generar descripciones y/o resúmenes de lo que está ocurriendo.

2. Objetivos de aprendizaje

Al finalizar la Práctica 2, el estudiante debería ser capaz de:

- Reutilizar y empaquetar el *pipeline* de clasificación de la Práctica 1 como un módulo reusable.
- Implementar un módulo de *retrieval* denso (k-NN sobre *embeddings*) sobre el conjunto de entrenamiento.
- Construir un clasificador basado en vecinos y un clasificador híbrido (Transformer + k-NN) y comparar su rendimiento.
- Utilizar *retrieval* como mecanismo de explicabilidad: mostrar ejemplos soporte de cada decisión.
- Comparar un modelo Transformer “teacher” frente a un modelo comprimido/destilado (tamaño, velocidad, rendimiento).
- Aplicar resumen abstractivo para generar:
 - Descripciones globales de cada clase.
 - Explicaciones locales basadas en vecinos recuperados.
- Analizar el *trade-off* calidad $\leftrightarrow$ coste computacional, y reflexionar sobre sesgos, alucinaciones y limitaciones del sistema.

3. Material de partida

Se debe partir del repositorio de la Práctica 1, donde se debe encontrar:

- Código para:
 - Cargar y preprocesar el *dataset* armonizado.
 - Entrenar y evaluar el modelo Transformer “teacher”.
- El mejor modelo Transformer que hayáis obtenido (o, al menos, su configuración reproducible).
- Código de evaluación.

Antes de empezar la P2, se recomienda:

- Refactorizar mínimamente el código de P1 para exponer funciones reutilizables, por ejemplo:

```
def load_data():
    # Devuelve train_texts, train_labels,
    # val_texts, val_labels, test_texts, test_labels
    pass

def load_teacher_model():
    # Carga el modelo Transformer fine-tuneado de la P1
    pass

def predict_teacher(texts):
    # Devuelve logits/probabilidades y predicciones
    # del modelo de la P1
    pass
```

4. Tareas

4.1. Módulo de retrieval denso

Se implementa un módulo de retrieval independiente que:

Construcción del índice

- Usa el conjunto de entrenamiento de P1 como corpus:
 - X_{train} : textos.
 - y_{train} : etiquetas correspondientes.
- Define una función de *embedding* `embed_fn(texts)` que:
 - Usa el encoder del modelo Transformer de P1 (sin la última capa de clasificación).
 - Para cada texto, obtiene un vector d -dimensional (p. ej., salida de [CLS] o *pooling* de la última capa).

- Construye un índice k-NN:
 - Por ejemplo con `sklearn.NearestNeighbors` (métrica coseno) u otra biblioteca equivalente.

Búsqueda

- Se define una función `search(query_texts, k)` que, dada una lista de textos de consulta, devuelve para cada uno:
 - Los k vecinos más cercanos en el espacio de *embeddings*.
 - La distancia / similitud.
 - Metadatos asociados (texto original, etiqueta, índice, etc.).

Evaluación básica de retrieval

- Para un conjunto de ejemplos de validación/test:
 - Calcular Precision@k y/o Recall@k de vecinos “con la misma etiqueta” que el ejemplo objetivo.
- Comentar brevemente:
 - ¿El espacio de *embeddings* separa bien las clases?
 - ¿Hay clases con vecinos sistemáticamente peores?

4.2. Clasificador k-NN sobre embeddings

Usando el índice anterior:

- Se tiene que definir un clasificador k-NN:
 - Dado un texto:
 - Obtener su *embedding* con `embed_fn`.
 - Recuperar los top- k vecinos de entrenamiento.
 - Predecir la clase del texto como voto mayoritario de las etiquetas de los vecinos.
- Evaluación de rendimiento:
 - Misma partición train/dev/test que en P1 (el k-NN sólo se entrena implícitamente con train).
 - Métricas mínimas:
 - Accuracy.
 - Macro-F1.
- Comparación:
 - k-NN vs TF-IDF + clasificador clásico de P1.
 - k-NN vs Transformer fine-tuneado de P1.
- Comentar brevemente:
 - ¿En qué clases va mejor/peor el k-NN frente al Transformer?
 - ¿Tiene sentido el comportamiento viendo los ejemplos?

4.3. Clasificador híbrido Transformer + k-NN (RAG para clasificación)

Se debe diseñar un clasificador híbrido que combine:

- Las probabilidades del Transformer de P1, $p_T(y | x)$.
- La distribución de clases en los vecinos, $p_K(y | x)$.

Una opción sencilla es:

$$p_{\text{comb}}(y | x) = \alpha p_T(y | x) + (1 - \alpha) p_K(y | x),$$

donde $0 \leq \alpha \leq 1$.

Tareas

- Implementar el clasificador híbrido:
 - Para varios valores de α (p.ej. 0.0, 0.25, 0.5, 0.75, 1.0).
 - Manteniendo k fijo (p.ej. $k = 5$ o $k = 10$).
- Evaluar:
 - Accuracy y Macro-F1 en el conjunto de test para distintos valores de α .
 - Comparar con:
 - Sólo Transformer (equivalente a $\alpha = 1$).
 - Sólo k-NN (equivalente a $\alpha = 0$).
- Comentar brevemente:
 - ¿Existe algún rango de α donde el híbrido mejore a ambos modelos por separado?
 - ¿Qué pasa con las clases minoritarias o más difíciles?

4.4. Explicabilidad basada en retrieval (ejemplos similares)

Se usa el módulo de retrieval para ofrecer explicaciones de ejemplo (*case-based reasoning*):

- Seleccionad al menos:
 - 10 ejemplos de test bien clasificados por el modelo híbrido.
 - 10 ejemplos de test mal clasificados.
- Para cada ejemplo seleccionado, se debe mostrar:
 - Texto original.
 - Predicción del híbrido y del Transformer solo.
 - Lista de los k vecinos recuperados (texto + etiqueta).
- Redactad un breve comentario (2–3 frases) del estilo:

«Este texto se parece mucho a estos otros 3 ejemplos etiquetados como X, por la presencia de expresiones como ‘...’, ‘...’, lo cual puede explicar por qué el modelo lo clasifica en la clase X».

- Comparad con las explicaciones de P1:
 - ¿Es más comprensible para un usuario final ver ejemplos similares que ver tokens/atributos importantes?
 - ¿Qué ventajas e inconvenientes tienen ambos enfoques?

4.5. Compresión / modelo destilado

En esta parte se debe comparar el Transformer de P1 con un modelo más pequeño.

- Elegid un modelo comprimido/destilado adecuado:
 - Si en P1 se usó algo tipo `bert-base`, ahora se puede usar: `distilbert-base-uncased`, `distilbert-base-multilingual-cased`, `MiniLM`, `ALBERT-base`, etc.
 - Debe ser claramente más ligero que el de P1 (menos parámetros).
- Entrenad/ajustad este modelo en el mismo *dataset* que en P1:
 - Mismo *split* train/dev/test.
 - Hiperparámetros razonables (se pueden partir de los de P1 y ajustarlos ligeramente).
- Evaluad y comparad:
 - **Calidad:**
 - Accuracy, Macro-F1 y per-class F1 en el test.
 - **Coste:**
 - Número aproximado de parámetros.
 - Tamaño en disco (MB) del modelo.
 - Tiempo medio de inferencia por lote (por ejemplo, 256 ejemplos) en la misma máquina.
- Análisis:
 - ¿Cuánta calidad se pierde al pasar de *teacher* → modelo comprimido?
 - ¿Cuánto se gana en velocidad/memoria?
 - ¿Se degrada más en algunas clases (p.ej. las minoritarias)?

4.6. Resumen automático como explicación

En este apartado se utiliza un modelo generativo (T5/BART pequeño u otro encoder–decoder/decoder-only apropiado) para generar:

- Resúmenes globales de cada clase.
- Explicaciones locales para ejemplos individuales, basadas en sus vecinos recuperados.

Resumen global por clase

- Para cada clase c :
 - Seleccionad $N \approx 50$ textos de entrenamiento de esa clase (por ejemplo, aleatorios o los mejor clasificados por el *teacher*).
 - Construid un “pseudo-documento” concatenando fragmentos de estos textos, separados por alguna marca clara (--).
- Usad un modelo de resumen abstractivo (p.ej. `t5-small`, `bart-base`) en modo inferencia:
 - **Input** (ejemplo):
`summarize: <pseudo-documento de la clase c>`
 - **Output**: un resumen de 3–5 frases que describa la clase.
- En el informe:
 - Mostrad el resumen generado para cada clase.
 - Comentad críticamente:
 - ¿Refleja bien qué caracteriza a la clase según vuestra experiencia?
 - ¿Detectáis sesgos o estereotipos en la descripción?
 - ¿Hay alucinaciones o frases demasiado vagas?

Resumen local basado en vecinos (explicación de predicciones)

- Seleccionad al menos 6 ejemplos de test:
 - 3 correctamente clasificados.
 - 3 erróneamente clasificados.
- Para cada uno:
 - Usad el módulo de retrieval para obtener sus k vecinos más cercanos de entrenamiento (texto + etiqueta).
 - Construid un texto de entrada del estilo:

Texto objetivo:

`<texto_test>`

Ejemplos similares de entrenamiento:

`[1] (<label1>): <texto_vecino1>`

`[2] (<label2>): <texto_vecino2>`

`...`

`[k] (<labelk>): <texto_vecinok>`

Escribe un breve resumen explicando qué tienen en común estos textos y por qué podrían pertenecer a la clase `<clase_predicha>`.

- Pasad esto al modelo de resumen/generación y guardad la explicación generada (2–3 frases).
- En el informe:

- Mostrad al menos 3 ejemplos completos (texto, vecinos y resumen explicativo).
- Valorad:
 - ¿Tiene sentido la explicación?
 - ¿Ayuda a entender el error/acierto del modelo?
 - ¿Se inventa cosas que no están en los textos? (alucinaciones).

4.7. Análisis global y discusión

En el informe debéis incluir una sección de discusión integrada, donde:

- Se compara:
 - Baselines de P1 vs k-NN vs híbrido vs modelo comprimido.
 - ¿Qué combinación de técnicas ofrece mejor equilibrio calidad $\leftrightarrow$ coste $\leftrightarrow$ interpretabilidad?
- Se analizan los aspectos de equidad y sesgo:
 - ¿Hay clases o tipos de texto sistemáticamente peor tratados por los modelos pequeños / híbridos?
 - ¿Podrían los vecinos agravar sesgos (p.ej. agrupar textos ofensivos de forma consistente)?
- Se reflexiona sobre:
 - Utilidad real de:
 - Explicaciones por tokens/atributos (P1).
 - Explicaciones por ejemplos (retrieval).
 - Explicaciones generativas (resumen).
 - Riesgos de:
 - Alucinaciones en los resúmenes/explicaciones.
 - Uso de modelos comprimidos sin reevaluar sesgos.

5. Tarea opcional (para MH)

Esta parte no es obligatoria, pero servirá como criterio para MH.

Knowledge Distillation “propia”

- Definir un *student* más pequeño y entrenarlo usando:
 - Loss sobre etiquetas reales.
 - KL entre las distribuciones del *teacher* y del *student* (con temperatura).
- Comparar *student* con y sin distillation.

6. Entregables

6.1. Informe

- Formato: PDF (`Informe_P2_GrupoX.pdf`).
- Extensión orientativa: 8–12 páginas (sin contar anexos).
- Estructura sugerida:
 - Introducción y resumen de la P1 (1/4–1/2 página).
 - Módulo de retrieval:
 - Definición, implementación, métricas.
 - Clasificador k-NN e híbrido:
 - Definición, resultados, análisis.
 - Compresión / modelo destilado:
 - Experimentos, tablas de calidad/coste.
 - Resumen automático (global y local):
 - Ejemplos, análisis de calidad y alucinaciones.
 - Discusión global:
 - Calidad vs coste, sesgos, explicabilidad, riesgos y posibles extensiones.
 - (Opcional) Resultados de las tareas bonus.
- Debe incluir:
 - Tablas y figuras legibles.

6.2. Código y notebooks

- Un `Notebook_P2.ipynb` o varios notebooks bien organizados con:
 - Construcción del índice y retrieval.
 - Experimentos k-NN e híbrido.
 - Entrenamiento y evaluación del modelo comprimido.
 - Generación de resúmenes globales y locales.
- Código fuente estructurado en un directorio (por ejemplo, `src/` o `pipeline/`), con:
 - Módulo de datos (reutilizando P1).
 - Módulo de modelos (teacher + comprimido).
 - `retrieval.py` o similar.
 - Scripts para entrenamiento/evaluación si se usan.

6.3. README

Archivo README_P2.md con:

- Cómo instalar dependencias.
- Cómo lanzar cada experimento (comandos/notebooks).
- Qué modelos se usan (teacher, comprimido, modelo de resumen).
- Tiempo aproximado de ejecución en Colab (o máquina similar).